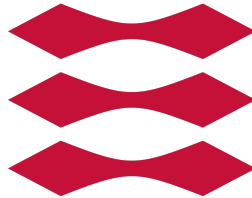


# DTU



---

Technical University of Denmark

## 02393 Programming in C++

---

Local Grader Guide

2024

---

**Authors:**

|                           |         |
|---------------------------|---------|
| Eszter Hetzmann           | s232532 |
| Frithjof Prochnow Sletten | s204477 |

---

# Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Windows</b>                                                  | <b>2</b> |
| 1.1      | (Recommended) Create an exam folder . . . . .                   | 2        |
| 1.2      | Download and unzip the zip folder in "C:\02393_exams" . . . . . | 2        |
| 1.3      | Folder overview . . . . .                                       | 2        |
| 1.4      | Modifying the Code . . . . .                                    | 3        |
| 1.5      | Running the Tests . . . . .                                     | 3        |
| <b>2</b> | <b>MacOS</b>                                                    | <b>5</b> |
| 2.1      | (Recommended) Create an exam folder . . . . .                   | 5        |
| 2.2      | Folder Structure and Contents . . . . .                         | 5        |
| 2.3      | Modifying the Code . . . . .                                    | 6        |
| 2.4      | Running the Tests . . . . .                                     | 6        |
| 2.4.1    | Fixing Permissions Issue . . . . .                              | 7        |
| <b>3</b> | <b>Linux(Ubuntu)</b>                                            | <b>8</b> |
| 3.1      | (Recommended) Create an exam folder . . . . .                   | 8        |
| 3.2      | Download and unzip the zip folder in " /02393_exams" . . . . .  | 8        |
| 3.3      | Folder overview . . . . .                                       | 8        |
| 3.4      | Modifying the Code . . . . .                                    | 8        |
| 3.5      | Running the Tests . . . . .                                     | 8        |

# 1 Windows

This guide assumes that you have installed g++ using [these instructions](#) and that you use the newest version of MSYS2 found at <https://www.msys2.org/>.

## 1.1 (Recommended) Create an exam folder

We recommend creating a folder for the exam located called "C:\02393\_exams". We recommend creating the folder to avoid having special characters such as space and åæø in the path.

## 1.2 Download and unzip the zip folder in "C:\02393\_exams"

1. Download the exam zip file and move it to "C:\02393\_exams".
2. Select the zip file and click "Extract all"
3. (optional) Update the path because the zip folder contains a folder with the same name.
4. Click "Extract"

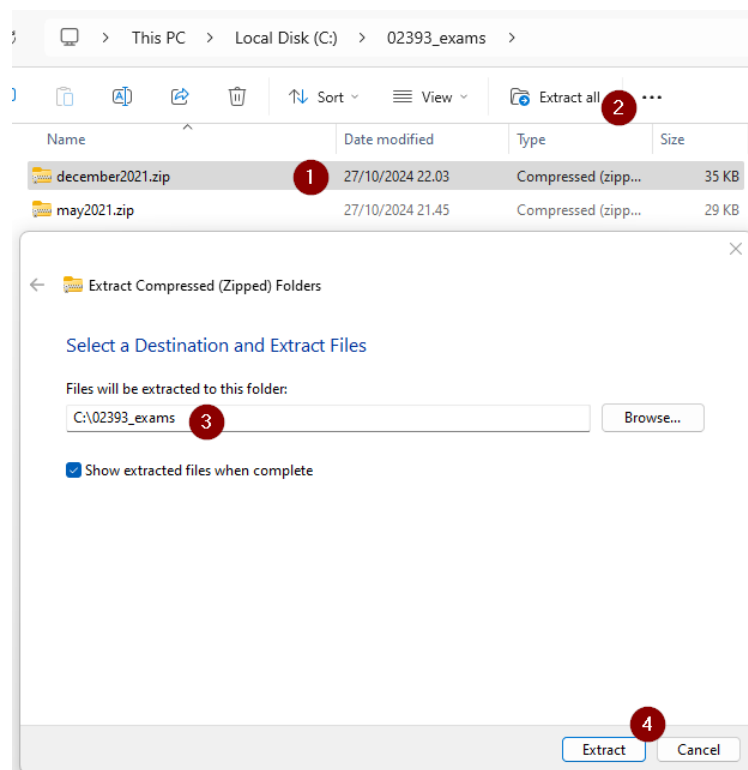


Figure 1: Extract zip files

## 1.3 Folder overview

Once you have decompressed the ZIP archives in the "02393\_exams" folder, you should see a set of folders named after each ZIP archive. Inside each of these folders, you will find subdirectories for each exercise. The structure of each exercise folder should contain the following files:

- **ex0X-library.cpp**: The skeleton of the C++ library file you need to implement.
- **ex0X-library.h**: The header file for the library.

- **ex0X-main.cpp**: A test program that uses the library.
- **runtest.sh**: A shell script to execute tests for the exercise.
- **test folder**: A folder containing test code and data.

## 1.4 Modifying the Code

For each exercise, you will need to modify the **ex0X-library.cpp** file to meet the requirements stated in the exam text. **If the instructions specify** any changes to the header file (**ex0X-library.h**), be sure to implement those as well.

## 1.5 Running the Tests

When you have solved an exercise. Open the MSYS2 UCRT64 terminal and **cd** to the folder for that exercise.

```
cd C:/02393_exams/<archive_name>/<exercise_folder>
```

For example the first exercise in the December 2021 exam.

```
cd C:/02393_exams/december2021/exercise1minichess
```

To do so, remember that the C: drive appears under the /c folder. To run the tests, type

```
sh runtest.sh
```

And if all goes well you will see passing tests as shown in figure 2. The results are also available in the file **results.txt**, where can see the expected result and what your program returned. As an example in figure 9 the threatened function has not been implemented correctly.

If your implementation has a bug that causes an infinite loop, **the script may hang** during execution. In this case, you will need to manually terminate the script. Use CTRL+C.

```

/c/02393_exams/december2021/exercise1minichess
Frith@tailow UCRT64 /c/02393_exams/december2021/exercise1minichess
$ cd C:/02393_exams/december2021/exercise1minichess
Frith@tailow UCRT64 /c/02393_exams/december2021/exercise1minichess
$ sh runtest.sh
Compiling Example
Running Example
Compiling TaskA
Running TaskA
Compiling TaskB
Running TaskB
Compiling TaskC
Running TaskC
Compiling TaskD
Running TaskD
tests completed!
results:
-----
Example: compiles: yes, passes tests: yes
-----
TaskA: compiles: yes, passes tests: yes
-----
TaskB: compiles: yes, passes tests: yes
-----
TaskC: compiles: yes, passes tests: yes
-----
TaskD: compiles: yes, passes tests: yes
-----
total score: 100%
Frith@tailow UCRT64 /c/02393_exams/december2021/exercise1minichess
$

```

Figure 2: Run tests using MSYS2 UCRT64

```
Example: compiles: yes, passes tests: no
reason: Example was expected to return the following output: Chessboard:
b _ r
_ _ _
R B _
Is the piece in (0,2) threatened? No!
Can we move from (0,0) to (1,0)? No!
Can we move from (2,0) to (2,1)? No!
Can we move from (2,0) to (0,0)? Yes!
Can we move from (2,1) to (1,2)? Yes!

The chessboard is now:
R _ r
_ _ B
_ _ _
Is the piece in (0,2) threatened? Yes!

Instead, it returned the following output:
Chessboard:
b _ r
_ _ _
R B _
Is the piece in (0,2) threatened? No!
Can we move from (0,0) to (1,0)? No!
Can we move from (2,0) to (2,1)? No!
Can we move from (2,0) to (0,0)? Yes!
Can we move from (2,1) to (1,2)? Yes!

The chessboard is now:
R _ r
_ _ B
_ _ _
Is the piece in (0,2) threatened? No!
```

Figure 3: Test fails because threatened has not been implemented correctly

## 2 MacOS

This guide assumes that you have installed clang using [these instructions](#) and [Visual Studio Code](#) on your computer.

### 2.1 (Recommended) Create an exam folder

We recommend creating a folder named "02393\_exams" in your home directory. You can do this by opening a Terminal window and typing in:

```
mkdir ~/02393_exams
```

After this you should see the folder created in your home directory, as shown in Figure 4.

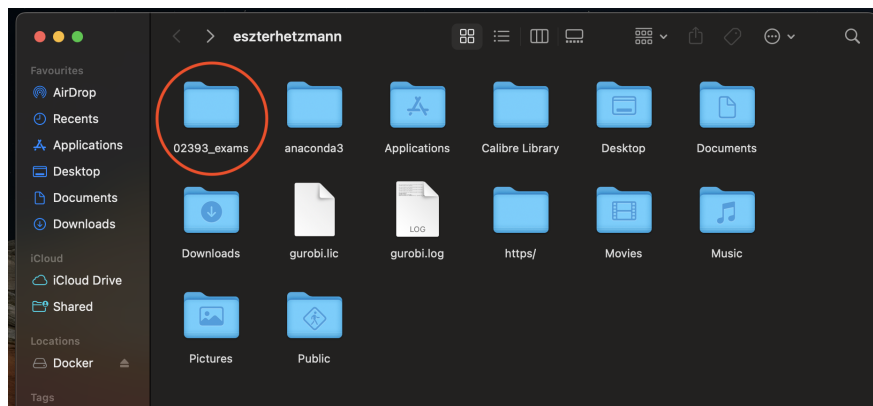


Figure 4: 02393\_exams Folder in your Home Directory

Now you can move the ZIP archives into that folder and decompress them, by right click and "Open", shown in Figure 5.

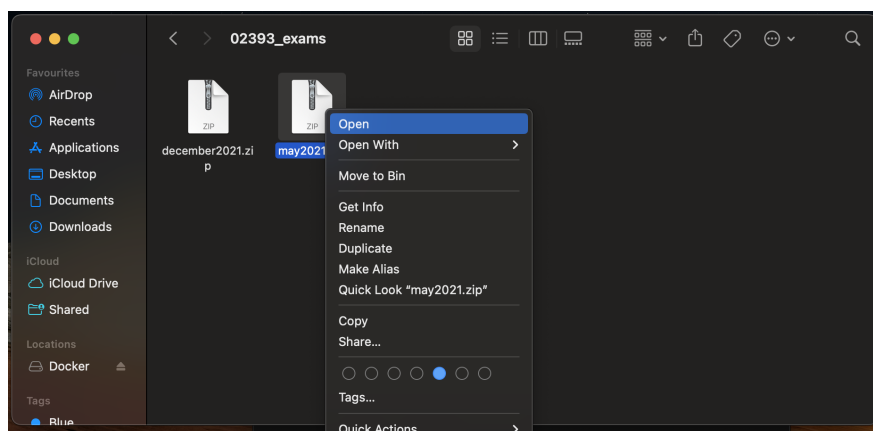


Figure 5: Decompressing the Zip Files

### 2.2 Folder Structure and Contents

Once you have decompressed the ZIP archives in the "02393\_exams" folder, you should see a set of folders named after each ZIP archive, as shown in Figure 6. Inside each of these folders, you will find subdirectories for each exercise. The structure of each exercise folder should contain the following files:

- **ex0X-library.cpp**: The skeleton of the C++ library file you need to implement.
- **ex0X-library.h**: The header file for the library.
- **ex0X-main.cpp**: A test program that uses the library.
- **runtest.sh**: A shell script to execute tests for the exercise.
- **test folder**: A folder containing test code and data.

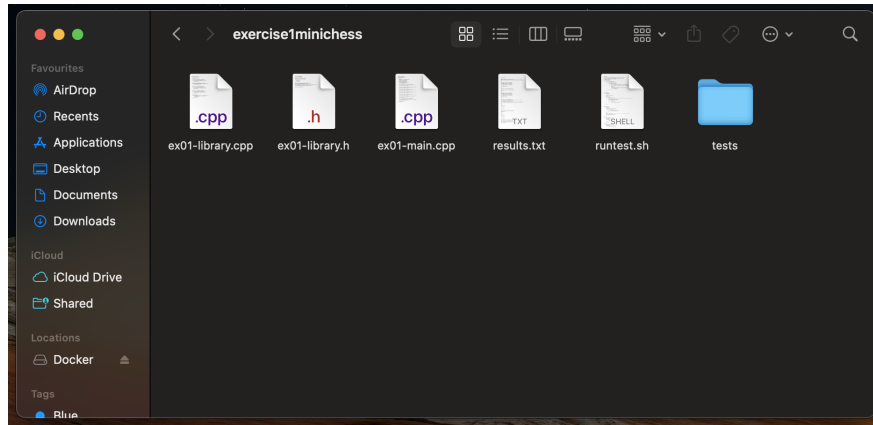


Figure 6: Folder Contents

## 2.3 Modifying the Code

For each exercise, you will need to modify the `ex0X-library.cpp` file to meet the requirements stated in the exam text. **If the instructions specify** any changes to the header file (`ex0X-library.h`), be sure to implement those as well.

## 2.4 Running the Tests

Once you’ve modified the necessary files, you can proceed to run the tests for each exercise:

1. Open a Terminal window.
2. Navigate to the specific exercise folder by typing:

```
cd ~/02393_exams/<archive_name>/<exercise_folder>
```

Replace `<archive_name>` and `<exercise_folder>` with the actual names of the archive and exercise folder. For example:

```
cd ~/02393_exams/may2021/exercise2rlelinkedlists
```

3. To run the tests, type the following command in the terminal:

```
sh runtest.sh
```

This will execute the tests, and you will see a report indicating the results of the tests. In case you run into a permission issue, check out Section 2.4.1.

If your implementation is correct, the output should match the one in Figure 7.

```

Compiling TaskD
Running TaskD

tests completed!

results:
-----
-n Example: compiles:
-n yes,
passes tests: yes
-----
-n TaskA: compiles:
-n yes,
passes tests: yes
-----
-n TaskB: compiles:
-n yes,
passes tests: yes
-----
-n TaskC: compiles:
-n yes,
passes tests: yes
-----
-n TaskD: compiles:
-n yes,
passes tests: yes
-----
-n total score:
-n 100

```

Figure 7: Test results: All tests passed

If there are errors in your implementation the report will show what output was produced by your code and the expected output, as shown in Figure 8. Additionally, you can find a detailed report in a file named `results.txt` within the exercise folder. Open this file to view a complete summary of the test outcomes and troubleshoot any errors. If your implementation has a bug that causes an infinite loop, **the script may hang** during execution. In this case, you will need to manually terminate the script. Use **Control-C**.

```

-----
-n TaskA: compiles:
-n yes,
passes tests: yes
-----
-n TaskB: compiles:
-n yes,
passes tests: yes
-----
-n TaskC: compiles:
-n yes,
passes tests: no
-n reason: TaskC was expected to return the following output:
The sum of the elements of an empty list is: 0
The sum of the elements of e0 is: 50
The sum of the elements of e1 is: 122
The sum of the elements of e2 is: 162
-----
Instead, it returned the following output:
The sum of the elements of an empty list is: 1
The sum of the elements of e0 is: 51
The sum of the elements of e1 is: 123
The sum of the elements of e2 is: 163
-----
-n total score:
-n 50

```

Figure 8: Test results: Some tests failed

### 2.4.1 Fixing Permissions Issue

If you encounter a `Permission denied` error when running the tests, it may be due to insufficient permissions on the exercise folder. To fix this, open a Terminal window and use the following command to grant read, write, and execute permissions to all files in your exercise folder:

```
chmod -R 755 ~/02393_exams/<archive_name>/<exercise_folder>
```

Replace `<archive_name>` and `<exercise_folder>` with the names of your specific archive and exercise folders. For example:

```
chmod -R 755 ~/02393_exams/may2021/exercise2rlelinkedlists
```

This command recursively changes permissions for all files and subfolders within the specified folder, allowing you to read, write, and execute these files without encountering permission issues.



## 3 Linux(Ubuntu)

This guide assumes that you use Ubuntu and that you have installed g++ using [these instructions](#) and installed unzip like:

```
sudo apt-get install unzip
```

### 3.1 (Recommended) Create an exam folder

We recommend creating a new folder where the path does not contain special characters such as space. In this guide will use

```
mkdir ~/02393_exams
```

### 3.2 Download and unzip the zip folder in " /02393\_exams"

1. Download the exam zip file and move it to " /02393\_exams".
2. Unzip the zip file (In this case the december2021 exam)

```
unzip ~/02393_exams/december2021.zip -d ~/02393_exams/
```

### 3.3 Folder overview

Once you have decompressed the ZIP archives in the "02393\_exams" folder, you should see a set of folders named after each ZIP archive. Inside each of these folders, you will find subdirectories for each exercise. The structure of each exercise folder should contain the following files:

- **ex0X-library.cpp**: The skeleton of the C++ library file you need to implement.
- **ex0X-library.h**: The header file for the library.
- **ex0X-main.cpp**: A test program that uses the library.
- **runtest.sh**: A shell script to execute tests for the exercise.
- **test folder**: A folder containing test code and data.

### 3.4 Modifying the Code

For each exercise, you will need to modify the **ex0X-library.cpp** file to meet the requirements stated in the exam text. **If the instructions specify** any changes to the header file (**ex0X-library.h**), be sure to implement those as well.

### 3.5 Running the Tests

When you have solved an exercise. Open a terminal and **cd** to the folder for that exercise.

```
cd ~/02393_exams/<archive_name>/<exercise_folder>
```

For example the first exercise in the December 2021 exam.

```
cd ~/02393_exams/december2021/exercise1minichess
```

To run the tests, type

```
bash runtest.sh
```

You will see the results printed in the terminal. The results are also available in the file **results.txt**, where can see the expected result and what your program returned. As an example in [figure 9](#) the threatened

function has not been implemented correctly. If your implementation has a bug that causes an infinite loop, **the script may hang** during execution. In this case, you will need to manually terminate the script. Use Control-C or Control-Z.

```
Example: compiles: yes, passes tests: no
reason: Example was expected to return the following output: Chessboard:
b _ r
_ _ _
R B _
Is the piece in (0,2) threatened? No!
Can we move from (0,0) to (1,0)? No!
Can we move from (2,0) to (2,1)? No!
Can we move from (2,0) to (0,0)? Yes!
Can we move from (2,1) to (1,2)? Yes!

The chessboard is now:
R _ r
_ _ B
_ _ _
Is the piece in (0,2) threatened? Yes!

Instead, it returned the following output:
Chessboard:
b _ r
_ _ _
R B _
Is the piece in (0,2) threatened? No!
Can we move from (0,0) to (1,0)? No!
Can we move from (2,0) to (2,1)? No!
Can we move from (2,0) to (0,0)? Yes!
Can we move from (2,1) to (1,2)? Yes!

The chessboard is now:
R _ r
_ _ B
_ _ _
Is the piece in (0,2) threatened? No!
```

Figure 9: Test fails because threatened has not been implemented correctly